

Real Example: Login & Profile

ResumeForge login, saving the token, and showing the profile



Dinesh Rawat

Instructor · Shorter Loop × Snapied

Let us use **ResumeForge** as our example. A user logs in, the server sends back a **token**, we save it, and then we use it to fetch the user's profile. This is the flow every real app uses. We keep the code simple: just call the API and subscribe. Angular 13 style.

1 The whole flow in one look

1. User types email and password in ResumeForge, clicks Login.
2. The app sends them to the server. The server replies with a **token**, which we save.
3. We move to the profile page.
4. The profile page asks the server for the user's data. The **interceptor** adds the saved token to that request.
5. The server checks the token and returns the user's profile, which we show.

2 The auth service (login and save the token)

Login sends the email and password with POST. When the token comes back, we save it. Simple and direct, no extra operators.

AUTH.SERVICE.TS

```
import { Injectable } from "@angular/core";
import { HttpClient } from "@angular/common/http";
import { Observable } from "rxjs";

@Injectable({ providedIn: "root" })
export class AuthService {
  private api = "https://api.resumeforge.com";

  constructor(private http: HttpClient) {}

  login(email: string, password: string): Observable<any> {
    return this.http.post(this.api + "/login", { email, password });
  }

  saveToken(token: string) {
    localStorage.setItem("token", token);
  }

  getToken() {
    return localStorage.getItem("token");
  }

  logout() {
    localStorage.removeItem("token");
  }
}
```

The service just makes the call and returns it. Saving is a separate small method. This keeps each method doing one clear job.

3 The login component

On submit, call login and subscribe. Inside the callback, save the token and go to the profile page. On error, show a message.

LOGIN.COMPONENT.TS

```
export class LoginComponent {
  email = "";
  password = "";
  error = "";

  constructor(private auth: AuthService, private router: Router) {}

  onLogin() {
    this.auth.login(this.email, this.password).subscribe(
      (res: any) => {
        this.auth.saveToken(res.token);    // save the token
        this.router.navigate(["/profile"]); // go to profile
      },
      (err) => {
        this.error = "Invalid email or password";
      }
    );
  }
}
```

LOGIN.COMPONENT.HTML

```
<input [(ngModel)]="email" placeholder="Email" />
<input [(ngModel)]="password" type="password" placeholder="Password" />
<button (click)="onLogin()">Login</button>
<p *ngIf="error">{{ error }}</p>
```

Why save the token here? Because the token only arrives when the server replies, which is inside the subscribe callback. That is the moment we have it.

Order matters, and it is safe here. Inside the callback, `saveToken` runs first and finishes immediately (it is a plain `localStorage.setItem`, not an API call), then `navigate` runs. So by the time the profile page loads and asks for data, the token is already saved. These two lines run one after the other, not at the same time.

4 The profile service (fetch the user)

This just asks for the user's profile. Notice it does **not** add the token by hand. The interceptor will do that.

PROFILE.SERVICE.TS

```
@Injectable({ providedIn: "root" })
export class ProfileService {
  private api = "https://api.resumeforge.com";

  constructor(private http: HttpClient) {}

  getProfile(): Observable<any> {
    return this.http.get(this.api + "/me");
  }
}
```

5 The interceptor (add the token to every request)

Every request passes through here. If we have a token, add it as a header. This is why the profile service did not need to.

AUTH.INTERCEPTOR.TS

```
@Injectable()
export class AuthInterceptor implements HttpInterceptor {
  constructor(private auth: AuthService) {}

  intercept(req: HttpRequest<any>, next: HttpHandler) {
    const token = this.auth.getToken();

    if (token) {
      const cloned = req.clone({
        setHeaders: { Authorization: "Bearer " + token },
      });
      return next.handle(cloned);
    }

    return next.handle(req);
  }
}
```

Register it in the module: `{ provide: HTTP_INTERCEPTORS, useClass: AuthInterceptor, multi: true }`. Now the profile request, and every request, carries the token automatically.

6 The profile component (show the user)

PROFILE.COMPONENT.TS

```
export class ProfileComponent implements OnInit {
  user: any = null;

  constructor(private profile: ProfileService) {}

  ngOnInit() {
    this.profile.getProfile().subscribe((data) => {
      this.user = data;
    });
  }
}
```

PROFILE.COMPONENT.HTML

```
<div *ngIf="user">
  <h2>{{ user.name }}</h2>
  <p>{{ user.email }}</p>
</div>
```

7 The routes (login and profile pages)

We keep sending the user to `/profile` after login, so we need routes that map those paths to their components. In Angular 13 you define them in the routing module.

APP-ROUTING.MODULE.TS

```
import { NgModule } from "@angular/core";
import { RouterModule, Routes } from "@angular/router";
import { LoginComponent } from "../login/login.component";
import { ProfileComponent } from "../profile/profile.component";

const routes: Routes = [
  { path: "login", component: LoginComponent },
  { path: "profile", component: ProfileComponent },
  { path: "", redirectTo: "login", pathMatch: "full" }, // default page
];

@NgModule({
  imports: [RouterModule.forRoot(routes)],
  exports: [RouterModule],
})
export class AppRoutingModule {}
```

Now `/login` shows the login page and `/profile` shows the profile page. The empty path redirects to login, so the app opens there. This is why `this.router.navigate(["/profile"])` in the login worked: the route exists.

You also need a `<router-outlet>` in your main template, the spot where the routed page appears:

```
<!-- app.component.html -->
<router-outlet></router-outlet>
```

Going further (guard): right now anyone can open `/profile` by typing the URL, even without logging in. A **route guard** can block that by checking for a token before letting the route open. We keep it simple today, but that is the next step to make it secure.

8 Styling it with SCSS

Now let us style the ResumeForge login and profile with **SCSS**. It is CSS with variables, nesting, and reuse, and Angular supports it out of the box. Component style files end in `.scss`.

LOGIN.COMPONENT.SCSS

```

$primary: #c0272d;
$radius: 6px;

.login-box {
  max-width: 320px;
  margin: 40px auto;
  padding: 24px;
  border-radius: $radius;

  input {
    width: 100%;
    padding: 10px;
    margin-bottom: 10px;
  }

  button {
    background: $primary;
    color: white;
    padding: 10px 16px;
    border: none;
    border-radius: $radius;

    &:hover {
      background: darken($primary, 10%);
    }
  }

  .error {
    color: $primary;
  }
}

```

Notice the wins over plain CSS: `$primary` stores the colour once, the `input` and `button` styles are **nested** inside `.login-box` to match the HTML, and `&:hover` means `button:hover`. Change `$primary` in one place and the whole page follows.

PROFILE.COMPONENT.SCSS

```

$primary: #c0272d;

.profile-card {
  max-width: 400px;
  margin: 40px auto;
  padding: 20px;
  border: 1px solid #eee;
  border-radius: 6px;

  h2 {
    color: $primary;
    margin-bottom: 4px;
  }

  p {
    color: #555;
  }
}

```

Tip: put shared values like `$primary` in one partial file, for example `_variables.scss`, and pull it into each component's styles. Then your whole ResumeForge app shares one colour palette.

Mixins: reusable blocks of styles

A **mixin** is a named group of styles you write once and drop into any rule with `@include`. Great for patterns you repeat, like centering or a card look.

```
@mixin flex-center {
  display: flex;
  align-items: center;
  justify-content: center;
}

.login-box {
  @include flex-center;
  flex-direction: column;
}
```

Mixins can take **arguments**, so one mixin makes many variations:

```
@mixin button($bg) {
  background: $bg;
  color: white;
  padding: 10px 16px;
  border-radius: 6px;
}

.save-btn { @include button(#c0272d); }
.cancel-btn { @include button(#6b7a8f); }
```

Inheritance with @extend

@extend lets one selector inherit all the styles of another, so you do not repeat them. If ResumeForge has a base button, other buttons can extend it and add their own bits.

```
.btn {
  padding: 10px 16px;
  border-radius: 6px;
  border: none;
}

.save-btn {
  @extend .btn; // takes all of .btn
  background: $primary;
}

.cancel-btn {
  @extend .btn;
  background: #6b7a8f;
}
```

Mixin vs @extend, the simple rule: use a **mixin** when you need arguments or fresh styles each time. Use **@extend** when several selectors truly share the same base look. Both cut repetition.

Operators and functions

SCSS can do maths and transform values, which plain CSS cannot:

```
.col {
  width: 100% / 3;           // maths
}

.button:hover {
  background: darken($primary, 10%); // colour function
}
```

Handy built-in colour functions include `darken()`, `lighten()`, and `rgba()`.

Placeholders (%): a lighter @extend

A **placeholder** starts with `%` and only appears in the final CSS if something extends it, so nothing is wasted:

```
%card-base {
  padding: 20px;
  border-radius: 6px;
  border: 1px solid #eee;
}

.profile-card { @extend %card-base; }
.resume-card { @extend %card-base; }
```

9 Remember this

Login sends email and password, gets a token back, and saves it in the subscribe callback.

The token proves who the user is. Protected requests must carry it.

The interceptor adds the token to every request, so the profile service never has to.

Routes map `/login` and `/profile` to their components, shown in a `<router-outlet>`. A guard can protect `/profile` later.

The payoff: log in once to get the token, and every protected call in ResumeForge just works.

SCSS styles it cleanly: `$variables` for colours, nesting to match the HTML, `@mixin` and `@include` for reusable blocks, and `@extend` for inheritance. Compiles to normal CSS.

10 Homework

Build this in ResumeForge: a login page that saves the token and moves to a profile page.

Add the interceptor so the profile request carries the token, and show the returned name and email.

Add a logout button that clears the token and returns to login.

Handle the error case so a wrong password shows a clear message.

Style it with SCSS: make a `$primary` variable, nest the button inside the login box, add a `&:hover`, write one `@mixin` for a button and use it twice, and try `@extend` on a base button.